

EDGE DA 10

NUMBER PLATE DETECTION PROJECT

25MML0014

ANUSREE V

The backend of the Number Plate Detection system is implemented using **Spring Boot with Java**, following the **MVC (Model–View–Controller) architecture**. The backend communicates with the frontend through **REST APIs**, which act as the entry point for all client requests. When a user captures or uploads a vehicle image from the frontend application, the image is sent to the backend using an **upload API**. This API receives the request and forwards it to the controller layer, which validates the input and passes it to the service layer for further processing.

The service layer performs the main processing operations of the system. Once the image is received, the backend first checks the request parameters and verifies whether the required services such as FTP connection and database services are active. After validation, the uploaded image is transferred to a **centralized FTP server**, which is used to store the images securely and make them accessible for further processing. Storing images in an FTP server reduces the load on the application server and enables efficient file management.

After the image is stored in the FTP server, the system initiates the **number plate detection process**. The detection model processes the image and identifies the number plate region by generating a bounding box around the license plate area. Once the number plate region is detected, **Optical Character Recognition (OCR)** techniques are used to extract the alphanumeric characters present on the plate. The extracted number plate text is then returned as the output of the detection process.

The system also maintains a **PostgreSQL relational database** to store system logs and detection results. The backend uses **Hibernate ORM** to interact with the database and manage SQL operations efficiently. Important information such as the image file name, upload time, detected number plate text, and processing status are stored in the database. These logs help track the processing steps and assist in debugging and monitoring the system.

The backend is designed to support **parallel processing**, allowing multiple users to access the system simultaneously. Spring Boot automatically manages multithreading, enabling the system to process multiple image upload requests without delay. This ensures that even when several users send images at the same time, the system can process them efficiently.

The entire backend application is packaged as a **JAR file**, which makes it easy to deploy and run on different systems. The application can be started using a simple Java command, which initializes the Spring Boot server, establishes database connections, and activates all API endpoints. Once the system starts successfully, it is ready to receive image upload requests and perform number plate detection operations.

Overall, the backend architecture provides a scalable and efficient framework for handling image uploads, processing vehicle number plates, storing logs, and supporting multiple concurrent users. The integration of **Spring Boot, FTP server, PostgreSQL database, and machine learning-based detection** ensures that the system operates reliably and efficiently for real-time number plate detection tasks.

Number Plate Detection Backend Workflow

1. Client / Frontend Request

- The process starts from the **frontend application** (camera system / UI / web app).
- The frontend **captures the vehicle image** containing the number plate.
- The image is sent to the backend through an **API request**.

Example API:

POST /upload

The API acts as the **entry point where frontend communicates with backend**.

2. API Layer (Spring Boot Controller)

Spring Boot provides REST APIs.

Controller responsibilities:

- Receive the uploaded image
- Validate request
- Pass request to service layer

Example flow:

Client → API Controller → Service Layer

Controller checks:

- File format
- File size
- Request validity

3. Service Layer Processing

The **Service class** handles the main business logic.

Operations performed:

1. Validate uploaded image
2. Generate metadata (timestamp, filename)
3. Send image to FTP server
4. Trigger number plate detection model
5. Store logs in database

4. Image Storage (FTP Server)

Uploaded images are stored in an **FTP server**.

Purpose:

- Store images centrally
- Make them accessible for processing
- Reduce load on application server

Workflow:

Spring Boot → FTP Server
 → Upload image

Stored information:

- image name
- upload time
- file path

5. Model Processing (Number Plate Detection)

After image upload:

The **ML model processes the image**.

Steps:

1. Load image from FTP server
2. Run detection model
3. Identify vehicle number plate
4. Extract text using OCR

Example pipeline:

Image → Detection Model → Bounding Box
→ OCR → Number Plate Text

Output example:

KL07AB1234

6. Database Storage (PostgreSQL)

All results and logs are stored in **PostgreSQL (RDBMS)**.

Stored data includes:

Field	Description
filename	uploaded image name
upload_time	timestamp
detected_plate	extracted number
processing_status	success/failure
server_logs	system logs

Spring Boot uses **Hibernate (ORM)** to interact with the database.

7. Logging System

Logs are generated to track system activities.

Logs include:

- API request received
- image uploaded
- detection status
- errors

Example log:

Image uploaded successfully
Detection completed
Plate detected: KL07AB1234
Logs help in **debugging and monitoring**.

8. Parallel Processing (Multiple Users)

Spring Boot supports **automatic multithreading**.

This means:

- Multiple users can call API simultaneously
- Image processing happens **in parallel**
- No delay even with multiple requests

Example:

User1 → Upload Image

User2 → Upload Image

User3 → Upload Image

All requests processed simultaneously

9. Response to Client

After processing:

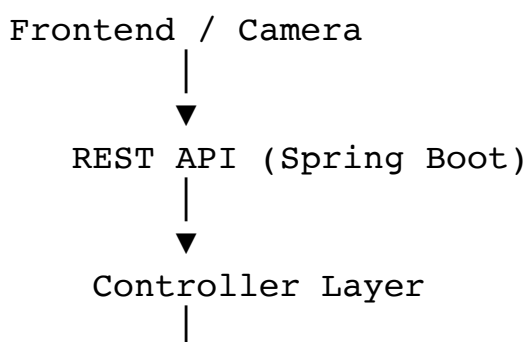
The backend sends response to frontend.

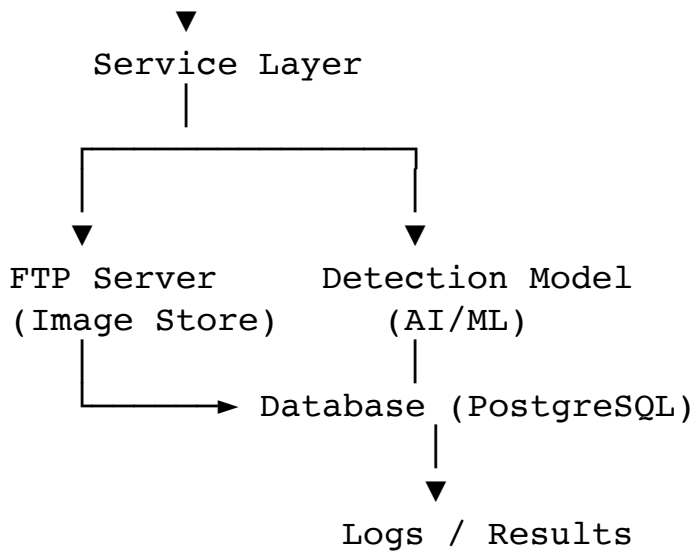
Example response:

```
{
  "filename": "vehicle1.jpg",
  "plate_number": "KL07AB1234",
  "status": "success",
  "upload_time": "2026-03-06 10:21:30"
}
```

10. System Architecture Overview

Overall architecture:





11. Deployment

The backend application is packaged as a **JAR file**.

Run command:

```
java -jar number-plate-detection.jar
```

When started:

- Spring Boot server initializes
- Database connection established
- FTP connection configured
- APIs become active

